Національний технічний університет України «КПІ ім. Ігоря Сікорського»

Лабораторна робота №3
з дисципліни «Розробка мобільних застосувань під Android»

на тему
«ДОСЛІДЖЕННЯ СПОСОБІВ ЗБЕРЕЖЕННЯ ДАНИХ»

Виконав:
студент групи ЗПІ-ЗП-31
Сікорський О.С.

Викладач:
к.т.н.
Орленко С.П.

Київ – 2026

Мета роботи: дослідити способи збереження даних (база даних, файлова система, тощо) та отримати практичні навички щодо використання сховищ даних.

Завдання: Написати програму під платформу Андроїд, яка доповнює програму, що розроблена за лабораторною роботою 2, роботою зі сховищами. Тобто при натисканні на кнопку «ОК» додатково: - здійснюється запис результату взаємодії з інтерфейсом до сховища (файл або базу даних); - користувач інформується відповідним повідомленням щодо успішності запису. Також інтерфейс необхідно доповнити кнопкою «Відкрити», натискання на яку призводить до переходу на іншу Діяльність, у якій відображається вміст даних, що зберігаються у сховищі. Якщо дані відсутні (сховище пусте) відобразити відповідне повідомлення. За бажанням можна додатково реалізувати оновлення та видалення даних зі сховища.

Виконання:

Роботу:
- виконано в середовищі Android Studio з використанням Android SDK 36;
- протестовано на реальному пристрої Lenovo Tab P12 (Android 15).

Під час виконання лабораторної роботи №3 застосунок, розроблений у попередній роботі, було розширено функціоналом для постійного збереження, перегляду, оновлення та видалення даних.

1. **Реалізація сховища даних:** В якості локального сховища було обрано реляційну базу даних **SQLite**, управління якою здійснюється за допомогою класу DatabaseManager. Для зручності тестування реалізовано автоматичне наповнення бази даних при першому запуску з підготовленого файлу assets/books.csv.
2. **Запис результатів та інформування:** Збереження нового запису до сховища ініціюється при натисканні кнопки **"Save"** (яка виконує роль кнопки підтвердження) у діалоговому вікні додавання книги. Дані з полів (назва, автор, рік) передаються до БД, після чого користувач інформується про успішність операції за допомогою Toast повідомлення з текстом "Added: [Назва книги]", виклик якого координується через MainActivity.
3. **Перегляд збережених даних:** Для перегляду вмісту сховища замість переходу на нову Діяльність (Activity) було застосовано більш

сучасний архітектурний підхід з використанням **ViewPager2** та **TabLayout**. Дані відображаються на двох вкладках:
    - **"Library"** — виводить список усіх збережених книг за допомогою RecyclerView.
    - **"Statistics"** — відображає агреговану інформацію (загальна кількість записів, кількість прочитаних та непрочитаних книг).
    - Якщо сховище порожнє, RecyclerView залишається пустим, а статистика відображає нульові значення.
4. **Додатковий функціонал (Оновлення та Видалення):** Згідно з рекомендаціями до завдання, реалізовано можливість змінювати статус книги (прочитано/не прочитано) безпосередньо в елементі списку, а також видаляти книгу з БД натисканням на відповідну іконку.

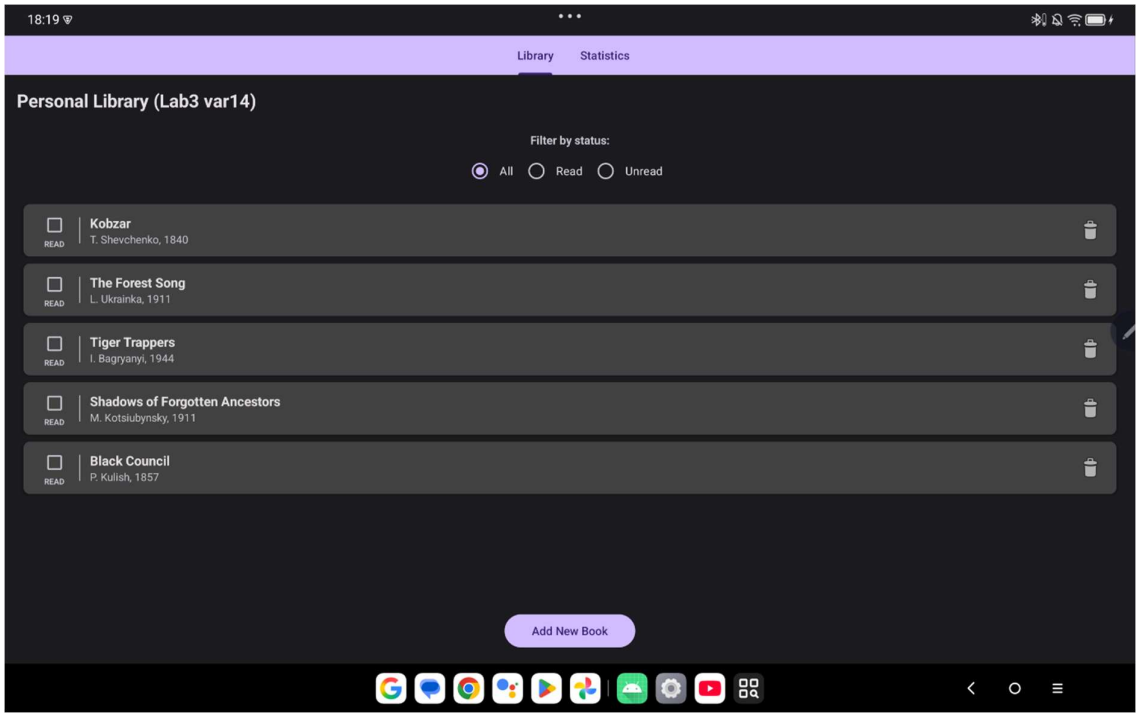
Ключові класи та методи:

Оскільки цей розділ є найважливішим для розуміння логіки роботи, нижче наведено детальний опис методів, що відповідають за роботу зі сховищем та навігацією:

- **У класі** DatabaseManager **(центральний клас для роботи з SQLite БД):**
    - onCreate() — метод життєвого циклу БД, який виконує SQL-запит для створення структури таблиць.
    - seedDatabaseFromCSV() — метод, що зчитує початкові дані з файлу в теці assets та автоматично імпортує їх до бази даних.
    - insertBook(book: Book) — здійснює запис нового об'єкта книги до таблиці books.
    - getAllBooks() — робить вибірку з БД та повертає повний список усіх збережених книг для відображення у списку.
    - updateBookStatus(id, isRead) — метод для оновлення стану запису в базі (зміна статусу прочитання) за його ідентифікатором.
    - deleteBook(id) — виконує видалення конкретного запису зі сховища за його ідентифікатором.
- **Модель даних** Book**:**
    - Data-клас, який описує сутність книги та містить поля: id, title, author, year, isRead.
- **Методи навігації та координації:**
    - У класі MainActivity (хост-активність) реалізовано методи для координації взаємодії між фрагментами та управління виведенням сповіщень Toast при успішних транзакціях у БД.

- Клас LibraryPagerAdapter містить логіку адаптера, що забезпечує роботу ViewPager2 для плавного перемикання між фрагментами списку книг (Library) та їх статистики (Statistics).

Загальний вигляд інтерфейсу додатка:



Висновок: У ході виконання лабораторної роботи було успішно досліджено та застосовано на практиці механізми збереження даних у мобільних додатках Android за допомогою локальної бази даних SQLite. Було реалізовано повноцінний CRUD-функціонал (створення, читання, оновлення, видалення записів). Крім того, навички роботи з інтерфейсом користувача були покращені завдяки застосуванню RecyclerView для виводу даних зі сховища та ViewPager2 з TabLayout для зручної навігації. Мета роботи досягнута, технічне завдання виконано в повному обсязі з додаванням додаткового функціоналу.

Відповіді на контрольні питання:

Питання: Опишіть як організована робота з налаштуваннями (даними у вигляді пари «ключзначення»).

Відповідь: Для збереження відносно невеликого набору пар "ключ-значення" використовуються API-інтерфейси SharedPreferences,. Щоб створити новий файл налаштувань або отримати доступ до існуючого,

необхідно викликати метод getSharedPreferences() (якщо потрібно декілька файлів, які ідентифікуються за назвою) або getPreferences() (якщо потрібен лише один файл за замовчуванням для поточної Діяльності),. Для запису даних потрібно створити об'єкт типу SharedPreferences.Editor за допомогою методу edit(), зберегти потрібні значення (наприклад, через putInt(), putString()) та обов'язково викликати метод commit() для збереження змін,. Для отримання даних з файлу налаштувань викликаються методи на зразок getInt() та getString(), куди передається ключ і значення за замовчуванням, яке повертається, якщо ключ відсутній.

Питання: Опишіть типи сховищ файлів та причини їх використання.
Відповідь: Всі пристрої Android мають дві області зберігання файлів: внутрішня пам'ять (Internal storage) та зовнішні сховища (External storage). Внутрішня пам'ять завжди доступна, а файли в ній за замовчуванням доступні лише вашому застосунку і автоматично видаляються при видаленні програми,. Її найкраще використовувати для ситуацій, коли необхідно гарантувати, що інші програми або користувач не матимуть доступу до ваших файлів. Зовнішнє сховище (наприклад, SD-картка) доступне не завжди, не має контролю читання даних, і файли з нього видаляються разом із застосунком лише у випадку їх збереження у спеціальній директорії через getExternalFilesDir(),. Воно найкраще підходить для файлів без обмежень доступу, які повинні бути загальнодоступними для інших програм або користувача.

Питання: Опишіть процес роботи з файлами та файловою системою.
Відповідь: Файлова система Android побудована на основі Linux, тому використовує прямий слеш / як розмежовувач, а назви каталогів та файлів є реєстрозалежними. Програма зазвичай зберігає свої дані в каталозі /data/data/<назва_пакета>/, а створені файли потрапляють у підкаталог /files,. Для роботи з файлами абстрактний клас Context надає методи openFileInput(), openFileOutput(), getFilesDir(), getCacheDir() та deleteFile(),. Для безпосереднього читання та запису даних також застосовуються стандартні класи Java пакета java.io, зокрема FileInputStream та FileOutputStream. Для запису або читання файлів у зовнішньому сховищі необхідно попередньо прописати дозволи WRITE_EXTERNAL_STORAGE та READ_EXTERNAL_STORAGE у файлі маніфесту,.

Питання: Опишіть процес роботи з базами даних SQLite за допомогою класу SQLiteOpenHelper, наведіть переваги та недоліки.
Відповідь: Процес роботи починається зі створення підкласу SQLiteOpenHelper, в якому обов'язково перевизначаються методи onCreate(), onUpgrade() та onOpen(). Система не створює та не оновлює БД відразу при запуску програми, а відкладає ці довготривалі операції до

виклику методів getWritableDatabase() або getReadableDatabase(),. Дані додаються за допомогою методу insert() і об'єкта ContentValues, читаються через query() (який повертає Cursor), оновлюються методом update() та видаляються методом delete(),,,. *Переваги:* Збереження даних у БД ідеально підходить для структурованих і повторюваних даних. Крім того, використання механізму передачі параметрів через аргументи вибору (наприклад, у методі delete()) захищає БД від SQL-ін'єкцій. *Недоліки:* Безпосереднє використання API SQLite вимагає багато шаблонного коду, який часто повторюється і схильний до помилок, а також не підтримує перевірку SQL-запитів на етапі компіляції (саме тому рекомендується перехід на бібліотеку Room).

Питання: Як відбувається робота з БД за допомогою бібліотеки Room, наведіть переваги та недоліки.

Відповідь: Архітектура Room складається з трьох основних компонентів: класу бази даних (анотованого @Database), сутностей (анотованих @Entity), що описують таблиці, та об'єктів доступу до даних DAO (анотованих @Dao), які містять SQL-запити,. Клас бази даних надає програмі екземпляри DAO, через які відбувається взаємодія із сутностями даних для читання, оновлення чи вставлення рядків у таблиці. *Переваги:* Room простий у використанні, підтримує перевірку SQL-запитів під час компіляції, містить зручні анотації, що мінімізують шаблонний код, пропонує оптимізовані шляхи міграції бази даних та легко інтегрується з RxJava та LiveData. *Недоліки:* У наданих матеріалах не зазначено недоліків використання бібліотеки Room порівняно зі звичайним SQLite.

Питання: Наведіть характеристики екранів мобільних пристроїв.

Відповідь: Екран мобільного пристрою характеризується такими параметрами:

- Розмір (діагональ) екрану, що вимірюється в дюймах.
- Роздільна здатність (розширення) — кількість активних точок, що формують картинку.
- Показник щільності пікселів (dpi) — кількість точок на дюйм.
- Технологія виробництва дисплея.
- Тип конструкції «тачскріна» (сенсорного покриття).

Питання: Наведіть класифікацію та відмінності технологій (типів) екранів мобільних пристроїв.

Відповідь: Екрани поділяються на дві основні категорії: на основі LCD та на основі OLED.

- **LCD (рідкокристалічні) дисплеї** потребують заднього підсвічування і поділяються на пасивні (TN, CSTN, FSTN, DSTN, UFB) та активні матриці (TFT, IPS, PLS),,,,. TN-матриці дешеві і

мають низьке енергоспоживання, але відрізняються поганими кутами огляду, низькою якістю і швидкістю оновлення картинки,. Активні матриці (TFT, IPS) мають високу контрастність, широкі кути огляду (до 178°) та точну передачу кольорів (ідеальний білий), але споживають більше енергії,,,.

- **OLED дисплеї** (OLED, AMOLED, Super AMOLED) не потребують підсвічування, адже використовують органічні світлодіоди,. Вони значно тонші (близько 1 мм), споживають менше енергії (порівняно з РК), мають кути огляду до 180°, кращу насиченість та контрастність,,. Головним недоліком OLED є відносно низький час життя діодів (5-8 тисяч годин) і вигоряння пікселів,.

Питання: Наведіть поняття та характеристику сенсорних екранів.
Відповідь: Сенсорний екран (тачскрін) — це елемент конструкції (покриття) дисплея, який реагує на торкання. Наприклад, інфрачервоні сенсорні екрани працюють за рахунок переривання інфрачервоного променя при торканні, що дозволяє контролеру точно обчислити координати дотику. Вони характеризуються високим розширенням, довговічністю та не спотворюють зображення, проте є чутливими до пилу, конденсату та не мають антивандальних властивостей.
Питання: Наведіть загальну класифікацію сенсорних екранів.
Відповідь: Згідно з наданими матеріалами, виділяють інфрачервоні сенсорні екрани, а також технології, що використовують п'єзоелектричні перетворювачі та відбивачі звукових хвиль. *(Примітка: іншої загальної класифікації у джерелах не наведено).*
Питання: Наведіть рекомендації щодо розробки інтерфейсів для сенсорних екранів.
Відповідь: При розробці GUI для сенсорних екранів необхідно підганяти розміри елементів так, щоб їх було зручно читати зі звичної відстані, враховуючи клас пристрою (маленькі пристрої часто використовують на ходу, а більші тримають далі від очей). Крім того, слід завжди враховувати похибки при натисканні на екран: необхідно збільшувати цільову область торкання та робити кнопки і поля достатньо великими, щоб користувач міг легко потрапити у ціль навіть при неточному торканні.